The final option, stand-alone mode, saves time by using Bundler only when you install gems. At runtime, you're just using plain Ruby, with Bundler cut completely out of the equation. To use this mode, pass the --standalone option to Bundler:

```
$ bundle install --standalone
```

This command generates a file in your project directory called bundle/bundler/setup.rb, which sets up Ruby's $LOAD_PATH with the exact gem versions your project is configured to use. All you have to do is require this file from your code, *before* loading any gems. In fact, you can even skip this require step by using the --binstubs option together with --standalone:

```
$ bundle install --standalone --binstubs
```

This command generates *binstubs*—wrappers around your gems' Ruby commands like rspec and rake—inside your project's bin directory. Each of these will load bundle/bundler/setup.rb for you, and then run its original command.

This technique allows RSpec to start noticeably more quickly, especially in large projects—which gives us near-instantaneous feedback when running individual spec files. You will, however, need to remember to rerun this command any time your Gemfile or Gemfile.lock changes. We recommend creating an alias in your shell, such as bisb, and running it when you add or remove gems, pull code from someone else, change branches, and so on.

The improvement is particularly dramatic when you use the technique from *Ensuring the Application Works for Real*, on page 98 to run each of your spec files in isolation. Here's how long that task took using regular bundle exec:

```
$ time (for file in spec/**/*_spec.rb                ↩
          do bundle exec rspec $file || exit 1       ↩
        done) > /dev/null

1.50s user 0.17s system 98% cpu 1.707 total
```

Here's the same loop, but using the stand-alone-aware binstub instead:

```
$ time (for file in spec/**/*_spec.rb                ↩
          do bin/rspec $file || exit 1               ↩
        done) > /dev/null

0.85s user 0.11s system 97% cpu 0.983 total
```

From 1.7 seconds to less than a second! Stand-alone mode is nearly twice as fast, because the boot time for each rspec invocation is faster. On a project with a lot of gems (or a lot of spec files!), the difference is even more dramatic.